Contents

SPECIFICATION — Bounded Research-Agent Runtime (agentic workflows, tools as APIs, authorization, budgets, traces, research-agent + uv)

- **Status:** v0.1 — draft for implementation review (targeting Level 3).
- **Language:** Python 3.12 | Decision policy: local Ollama `qwen3.8:27b-mlx` (deterministic `MockPolicy` offline) | Tools: deterministic local-corpus `search` / `retrieve` | Schema: json-schema | Config: PyYAML | HTTP: httpx | GUI: PyQt5 (optional)

- **Curriculum source:** `curriculum/week1/chapter5.md` (§1 The Evolution from LLM Calls to Agents, §2 Stage 2: LLM + Tools, §3 Tool Calling Is an API Contract, §4 Agents vs. Workflows, §5 The Agent Loop, §6 State Is the Agent's Memory, §7 Observation Is Different from Reasoning, §8 Planning, §9 Reflection, §10 Loops and the Problem of Non-Termination, §11 Stopping Conditions, §12 Retries, §13 Failure Recovery, §14 Delegation, §15 Permissions: The Agent Is Not the User, §16 Human-in-the-Loop, §17 A Small Research Agent, §18 The Agent Runtime, §19 The Agent Prompt, §20 The Agent Trace, §21 Deliberately Breaking the Agent, §22 Failure 4: Tool Returns Misleading Data, §23 Failure 5: Infinite Loop, §24 Failure 6: Contradictory Sources, §25 Failure 7: Permission Violation, §26 Agent Reliability Is a Systems Problem, §27 Observability, §28 Deterministic Infrastructure Around a Probabilistic Core, §29 Agentic Workflow vs. Autonomous Agent, §30 The Agent Design Spectrum, §31 A More Robust Agent Runtime, §32 The Core Engineering Insight, §33 Engineering Principles, §34 Chapter 5 Laboratory, §35 What You Should Understand by the End of Chapter 5, §36 Key Takeaways).
- **Scope of this document:** the *authoritative specification* of the ch5 **agent runtime subsystem** — the deterministic control loop (state, budgets, stopping conditions, retries, authorization, validation, traces) wrapped around one probabilistic decision policy, plus the §34 drill harness that deliberately breaks the agent and grades the recovery. It is written to Level 2–3 (structured, mostly executable): behavior, interfaces, invariants, edge cases, and failure semantics are made explicit so an agent (or engineer) can derive implementation **and** verification with minimal inference.
- **Normative language:** `MUST`, `MUST NOT`, `SHALL`, and `SHALL NOT` are normative. `SHOULD` denotes a strong recommendation; `MAY` an optional behavior.
- **Principle:** ch5 §28 — *"Put probabilistic behavior where judgment is valuable; put deterministic mechanisms around it where correctness is required."* The LLM is the system's **policy** ($a_t \sim \pi_\theta(a \mid s_t)$, §4/§32); every mechanism that controls external side effects — schemas, permissions, budgets, retries, termination — is deterministic code, and the model is never the ultimate authority over any of them.

---

0. Intent and purpose

Chapter 5's central question is not "how do I build an agent?" but (chapter epigraph, verbatim):

> **"How do I build a bounded, observable, recoverable control loop in which a probabilistic model can safely perform useful work?"**

Chapters 1–3 built a fixed pipeline (prompt → context → retrieval → cited answer → judgment); chapter 4 made that pipeline measurable. Chapter 5 removes the predetermined execution graph: the model now *chooses the next transition* ($a_t \sim \pi_\theta(a \mid s_t)$, $s_{t+1} = T(s_t, a_t)$, §4), and the system becomes a **closed-loop control system** (§32). Everything ch1–ch4 taught about deterministic boundaries now applies to a moving target: the loop itself must be bounded (§10/§11), authorized (§15/§16), recoverable (§12/§13), and observable (§20/§27) — because the probabilistic policy *will* eventually do something unexpected (§33 principle 10).

This lab specifies the §34 exercise (*Chapter 5 Laboratory*): build a **research agent** with exactly two tools — `search(query)` and `retrieve(document_id)` — that accepts a research question, searches, retrieves sources, inspects the evidence, decides whether additional searching is necessary, and produces a final report (§17). Then **deliberately break it**: the §34 drill list (search timeout, empty results, malformed arguments, retrieval failure, duplicate searches, contradictory sources, low-quality sources, infinite-loop behavior, maximum-step exhaustion, unauthorized tool call) is run *through the runtime, not by eye*, and each drill is graded on the §34 four questions — what did the model do, what did the runtime do, what should have happened, what instrumentation would have exposed the problem.

The runtime instantiates §31's production loop (**budgets → decide → validate → authorize → execute-with-retry → update state**) around the same reliability split as ch1–ch4:

- **Deterministic boundary (pure, offline, reproducible, no LLM, no network):** the **runtime loop** and **state store** (§6), the **stopping conditions / budget enforcer** (§11), the **authorization policy engine** (§15), the **retry controller** with its failure-class taxonomy (§12), the **decision & argument validator** (§21 Failure 3, §31), the **tool router + tools** themselves (§3: tools are deterministic APIs over a local fixture corpus), the **trace writer** (§20/§27), the **loop metrics** (§33 principle 9), the **drill harness** (§34), and the **report writers**. Offline, the decision policy is the scripted, input-determined `MockPolicy` double (O-1), so the entire automated suite re-runs in CI without any model (ch3 R-17, carried).
- **Probabilistic boundary (unreliable component):** exactly one — the **decision policy** (`policy.py`): real Ollama generation (`qwen3.8:27b-mlx`, `/api/chat`) on the opt-in real path, the `MockPolicy` double otherwise. The policy *proposes* (`tool_call` | `final`); it never executes, authorizes, retries, or terminates. ch3's model-availability taxonomy (`DEGRADED_MOCK` / `PULL_REQUIRED` / `RUN_REAL`, ch3 E-13) is **carried over unchanged** so that "why did this run degrade to the mock policy" is never ambiguous in a trace (ch3 R-19 carried).

The termination discipline (§11) is the load-bearing invariant. The chapter's principle — *"Never rely on the model alone to decide when the system should stop"* (§11) — is encoded as a runtime-owned, closed set of stopping conditions (goal completion, max steps, token budget, cost budget, time budget, repeated-state detection, tool-failure threshold) evaluated **before every model call** (§31). The model *proposes* termination (`decision.type == "final"`); the runtime *enforces* it (I-001). Every run, adversarial or not, terminates with exactly one recorded `termination_reason` (I-011).

Authorization is outside the model (§15). Every proposed tool call passes the policy engine *before* execution: `read/search → allowed`, `delete_file → prohibited` (§25). The §25 experiment is a fixed acceptance drill: *"If the model can persuade the runtime to bypass authorization, the system is architecturally broken"* — the policy engine is code, not prompt, so there is nothing to persuade (I-002).

Uncertainty is a first-class outcome (§13). The agent's objective is not "always produce an answer" but *"produce the best answer justified by the available evidence, while accurately representing uncertainty and failure"* (§13). A run whose evidence is insufficient MUST end in a final report that states the limitation (`status: "insufficient_evidence"`) — that is **successful system behavior**, not a crash and never a hallucinated substitute (I-014; §21 Failure 2: absence of evidence is not evidence of absence).

Deployment decision: identical to ch3/ch4 — the real policy path is **local Ollama** at `http://localhost:11434`; when unavailable the runtime **degrades to the mock policy** and says so with ch3's exact E-13 banners. The runtime's *own* deterministic core is model-free (import/graph scan, T-02 analog of ch3 I-009/R-20, ch4 R-17).

Relationship to ch3/ch4. The ch3 RAG pipeline is a §4 **workflow** — a predetermined transition function $s_{t+1} = F(s_t)$; ch5 promotes retrieval to an **agentic loop** where the model selects actions, and §30's spectrum makes the tradeoff explicit. The two §34 tools are deliberately ch3-shaped (`search` ≈ retrieval, `retrieve` ≈ document fetch) so the *only* new variable is the loop itself. §33 principle 9 — *evaluate the loop, not just the final answer* — is the forward bridge to ch4: this lab emits per-run **loop metrics** (tool selection, argument correctness, unnecessary calls, recovery behavior, termination, cost, latency, §33.9/§27) as a versioned artifact that a ch4-style harness can consume.

Curriculum mapping: §3 → tool contracts (C-01/C-02); §5/§31 → the runtime loop (C-05, R-01); §6 → state model (C-04); §7/§20/§27 → trace records (C-07, R-11); §11 → stopping conditions (C-06, R-05); §12 → retry taxonomy (C-08, R-06); §13 → uncertainty outcomes (R-08, I-014); §15/§16/§25 → authorization (C-09, R-07); §17–§19 → research agent + prompt contract (R-02/R-03); §21–§25/§34 → the ten drills (C-11, R-10); §28 → the boundary split (above); §33 → engineering principles (invariants §6); §34 → laboratory acceptance (§9).

Primary product surface: a CLI (`research-agent`) with subcommands `run` (one bounded agent episode over the fixture corpus), `drill` (execute one §34 failure injection and emit the four-question drill report), and `trace` (render a saved trace human-readable), plus an optional PyQt5 GUI (`research-agent-gui`) that browses saved traces — never runs inference itself.

1. Actors and goals

Actor	Goals
User (human, single process)	Run one bounded research episode (mock or real policy); inject a §34 failure and read the drill report; render a saved trace; optionally browse traces in the GUI. (**single-principal** — no inter-principal authorization, ch3 F-009 / ch4 carried; the §15 policy engine governs *tool* authorization, not users.)
Policy / LLM (`policy.py`: `OllamaPolicy` real, `MockPolicy` offline double)	Given the serialized state + tool definitions + agent prompt (§19), emit exactly one decision per step: `{type: "tool_call", tool, arguments}` or `{type: "final", report}`. **Proposes only** — never executes tools, never self-authorizes, never terminates the loop (I-001/I-002). The `MockPolicy` is a documented, input-determined rule script (search → retrieve top hit → finalize; O-1) consuming no RNG (ch3 R-18 analog).
Runtime (`runtime.py`)	Own the §5/§31 control loop: initialize state, check stopping conditions *before every model call*, dispatch validate → authorize → execute-with-retry, update state, and guarantee termination with a recorded reason (I-001/I-011). Never itself calls the LLM API.
State store (`state.py`)	Hold the explicit §6 state $S_t = (G, H_t, O_t, M_t, P)$: goal, messages, observations, artifacts, step_count, budget counters, failure history. State lives here, **not** implicitly inside a prompt (§33 principle 3, I-010).
Tool router & tools (`tools.py`)	§3 API contracts: typed, schema-validated `search(query)` and `retrieve(document_id)` over a deterministic local fixture corpus; plus a registered-but-prohibited `delete_file(path)` existing solely to exercise §25 (C-02). Tools are deterministic: same input → same output, no network (I-003).
Validator (`validate.py`)	Reject malformed decisions/arguments *before* execution with the §21 structured error `{error, field, message}` fed back to the policy as a repair observation (R-04); validate the final report against its schema before acceptance (§31 `validate_final_answer`).
Authorization / policy engine (`authorize.py`)	§15: map every proposed `(tool, arguments)` to `allow` / `deny` from a declarative policy file (C-09); denial is final, logged, and returned as a structured observation — never retried (§12 Permission class, I-002).

Actor	Goals
Budget enforcer (`budgets.py`)	§11: evaluate the closed stopping-condition set (max steps, tokens, cost, time, repeated-state, consecutive-failures) before each step; any breach terminates the run with the matching `termination_reason` (C-06, I-001).
Retry controller (`retry.py`)	§12: classify every tool error into exactly one failure class (`TRANSIENT` / `INVALID_INPUT` / `AUTHENTICATION` / `PERMISSION` / `RATE_LIMIT` / `PERMANENT`) and apply the mapped strategy (bounded backoff-retry / repair / deny / abandon) — never blind-retry (I-006).
Tracer (`trace.py`)	§20/§27: append the full structured record per step (run id, model, prompt version, step, decision, tool, arguments, latency, result, tokens, cost, errors, retries) and the termination record; keep **action / observation / reasoning** as distinguishable typed entries (§7, I-004).
Drill harness (`drills.py`)	§34: inject one named failure (C-11 fault spec) into the tool layer, run the episode, and emit the four-question drill report (model behavior / runtime behavior / expected behavior / instrumentation) (R-10).
Loop metrics (`metrics.py`)	§33.9: compute per-run loop quality — steps used, tool-call mix, invalid-argument rate, retry counts, denial counts, unnecessary-call estimate, termination reason, latency, token/cost totals (R-11). Pure, headless, testable.
Ollama daemon *(external)*	Local runtime at `http://localhost:11434`: real policy generation (`/api/chat`, `qwen3.8:27b-mlx`). Not part of this project; ch3's E-13 taxonomy (carried) resolves its absence.
UI (`ui.py`, *optional*)	Browse a saved `trace.json` / `drill_report.json` offline (step timeline, decision/observation panes, termination banner); never blocks on inference; offscreen-testable (ch3 R-16 analog).

---

2. Requirements (intent, high level)

ID	Statement
R-01	The system shall execute the §5/§31 **agent control loop** — `initialize_state → [evaluate stopping conditions → policy decision → validate → authorize → execute-with-retry → update_state]* → validate final report → terminate` — where the **policy proposes and the runtime disposes**: the loop drives the policy, never vice versa. The runtime SHALL evaluate every stopping condition **before each model call** (§31), so a policy that never produces `final` still terminates (I-001).

ID	Statement
R-02	The **research agent** (§17/§34) SHALL expose exactly two operational tools — `search(query)` and `retrieve(document_id)` (C-01/C-02) — accept a research question, and produce a final report. A third tool, `delete_file(path)`, SHALL be **registered but prohibited** (C-09), existing solely as the §25 authorization target; it MUST NOT be executable under any prompt (I-002).
R-03	The **agent prompt** (§19) SHALL state the operational contract (may search/retrieve/analyze/report; do not invent evidence; prefer primary sources; state limitations; stop conditions). The prompt *specifies* behavior — including the step limit — but the **runtime enforces** every bound (§19: "the prompt specifies behavior, but the runtime still enforces the ten-call limit"); no behavioral guarantee SHALL rest on prompt text alone (I-001/I-002).
R-04	Every policy decision SHALL be **validated before execution** (§31 `valid_tool_call`): unknown tool, wrong argument types, or schema violation MUST be rejected with the §21 structured error observation `{"error": "invalid_arguments", "field": <name>, "message": <reason>}` fed back to the policy as a repair opportunity (C-03). A tool MUST never execute with invalid arguments (I-007).
R-05	The runtime SHALL enforce the §11 **closed stopping-condition set** — `goal_complete`, `max_steps`, `token_budget`, `cost_budget`, `time_budget`, `repeated_state`, `consecutive_tool_failures` — as a total enum (C-06). Each breach terminates the run with the matching `termination_reason`; the set is closed (no custom reasons) and every run ends with **exactly one** reason (I-011). *"The model can propose termination. The runtime should enforce it."* (§11).
R-06	**Retries** (§12) SHALL classify every tool error into exactly one failure class — `TRANSIENT` / `INVALID_INPUT` / `AUTHENTICATION` / `PERMISSION` / `RATE_LIMIT` / `PERMANENT` (C-08) — and apply the mapped strategy: `TRANSIENT` → bounded retry with deterministic backoff ($\leq$ `max_retries`); `RATE_LIMIT` → deterministic backoff, counted against the same bound; `INVALID_INPUT` → structured repair observation (R-04), no blind re-execution; `PERMISSION` → deny, **never retried**; `AUTHENTICATION` → terminate the tool path with escalation recorded; `PERMANENT` → abandon the tool, record, let the policy choose an alternative. Blind retry of every error is a violation (§12, I-006).
R-07	**Authorization is outside the model** (§15/§16): every proposed `(tool, arguments)` pair SHALL pass the declarative policy engine (C-09) *before* execution — `search`/`retrieve` → `allow`, `delete_file` → `deny`. A denial MUST be logged in the trace, returned to the policy as a structured `permission_denied` observation, and counted in loop metrics. No prompt content, tool argument, or policy output can alter the authorization map at runtime (I-002; §25: *"If the model can persuade the runtime to bypass authorization, the system is architecturally broken."*).
R-08	**Uncertainty is a first-class outcome** (§13): the final report SHALL carry `status: "ok" | "insufficient_evidence"`. When retrieved evidence cannot support an answer — including the empty-results case (§21 Failure 2: *absence of evidence is not evidence of absence*) — the report MUST state the limitation explicitly (the §13 behavior: *"The requested information could not be verified from the available sources."*) and MUST NOT fabricate claims or citations (I-014). An`insufficient_evidence` termination is **successful system behavior** (E-03).

ID	Statement
R-09	The **drill harness** (§34) SHALL execute the ten named failure drills of C-11 — `search_timeout`, `empty_results`, `malformed_arguments`, `retrieval_failure`, `duplicate_searches`, `contradictory_sources`, `low_quality_sources`, `infinite_loop`, `max_steps_exhaustion`, `unauthorized_tool_call` — by injecting a deterministic fault spec into the tool layer (never into the runtime itself) and emitting a `drill_report.json` answering the §34 four questions: model behavior / runtime behavior / expected behavior / instrumentation that exposed (or would expose) the problem (C-12). All ten drills MUST run fully offline on the `MockPolicy` (R-13).
R-10	**Non-termination defense** (§10/§23): the runtime SHALL detect semantic repetition — an `(tool, canonical(arguments))` pair seen `repeat_threshold` times — and terminate with `termination_reason: "repeated_state"` (C-06). A drill whose tool always yields nothing useful (§23) MUST end in `repeated_state` or `max_steps`, never in a hang (K-02).
R-11	**Observability** (§20/§27/§33.9): every run SHALL emit a versioned `trace.json` (C-07) carrying the full §27 field list — run id, question, model, model parameters, prompt version, per-step state snapshot reference, decision, tool name, arguments, tool latency, tool result, token usage, cost, errors, retries, `termination_reason`, final report — plus a **loop-metrics** row (C-10): steps used, tool-call mix, invalid-argument count, retry count, denial count, unnecessary-call estimate, latency, token/cost totals. *Evaluate the loop, not just the final answer* (§33 principle 9).
R-12	**Evidence evaluation** (§22/§24): observations SHALL carry provenance metadata (`source_id`, `quality: "primary" | "secondary" | "marketing"`, published date) separate from the model's interpretation (§7). The final report schema SHALL include a`conflicts` list: when two retrieved sources assert contradictory values for the same quantity (§24), the report MUST represent the conflict explicitly — never silently adopt the convenient value (E-08) — and MUST mark conclusions resting on `marketing`-quality sources with a`low_quality_evidence` caveat (E-09).
R-13	**Offline determinism** (ch3 R-17/R-18 carried): the *entire automated suite* runs offline via the `MockPolicy` double + fixture corpus; the mock path is **byte-identical for identical inputs** (floats at fixed `%.4f`, I-003), with **deterministic surrogates** for tokens/latency/cost derived from content lengths, explicitly labeled `usage_kind: "synthetic"` in the trace (ch4 R-14 analog). The `MockPolicy` consumes no seed/RNG — it is a pure function of `(state, tools)` (ch3 R-18 analog). The real Ollama path is opt-in/manual and best-effort.
R-14	**Model-availability taxonomy** (ch3 R-19/E-13 carried): on `--real` start the runtime resolves `DEGRADED_MOCK` (daemon unreachable → mock policy + banner, exit `0`), `PULL_REQUIRED` (model not pulled → exact `ollama pull <m>` remediation + exit `4`, never a crash), or `RUN_REAL` (pulled → real, no banner, exit `0`). Each outcome carries ch3's exact distinct banner text so a human never misreads why a mock policy ran (E-13).
R-15	An **optional PyQt5 GUI** (`research-agent-gui`, `MAY`) SHALL browse saved `trace.json` / `drill_report.json` artifacts offline — step timeline with typed action/observation/reasoning panes, budget gauges, termination banner, drill four-question view — without ever running inference (ch3 R-16 analog; offscreen-testable, T-13).
R-16	The **deterministic core** (`runtime.py`, `state.py`, `tools.py`, `validate.py`, `authorize.py`, `budgets.py`, `retry.py`, `trace.py`, `drills.py`, `metrics.py`, `report.py`) is **LLM- and network-free** — asserted by an import/graph source scan (T-02, ch3 I-009/R-20 analog). Only `policy.py` MAY import the LLM client (I-012).

ID	Statement
R-17	**Observation ≠ reasoning** (§7): trace entries SHALL be typed — `action` (the proposed call), `observation` (what the tool returned), `reasoning` (the policy's stated interpretation, when present). A tool result MUST be replayable verbatim from the trace; the policy's conclusions MUST NOT be written into observation records (I-004).
R-18	**Final-report validation** (§31 `validate_final_answer`): a `final` decision's report SHALL be schema-validated (C-03); an invalid report MUST NOT be accepted — the runtime appends a structured error observation and continues the loop (bounded by the same budgets, E-04). A run can therefore only end `ok`/`insufficient_evidence` on a *valid* report, or on a budget stop (I-011).
R-19	**Schema gate + artifact versioning** (ch3 R-19 analog): `trace.json` and `drill_report.json` SHALL each carry a literal version field (`agent_trace_version == "0.1"`, `drill_report_version == "0.1"`) and be validated against `schemas/*.json` on **every** load; malformed or version-mismatched artifacts are rejected deterministically (E-06). The `trace` subcommand and the GUI consume only validated artifacts (I-009).
R-20	**Bound everything, declaratively** (§33 principle 4): every bound — `max_steps` (default `10`, §19), `max_tokens`, `max_cost_usd`, `max_seconds`, `max_retries`, `repeat_threshold`, `max_consecutive_failures` — SHALL be settable in a `budgets.yml` config (C-06) validated on load (unknown keys are config errors, ch4 I-015 analog); documented defaults apply when no config is passed.

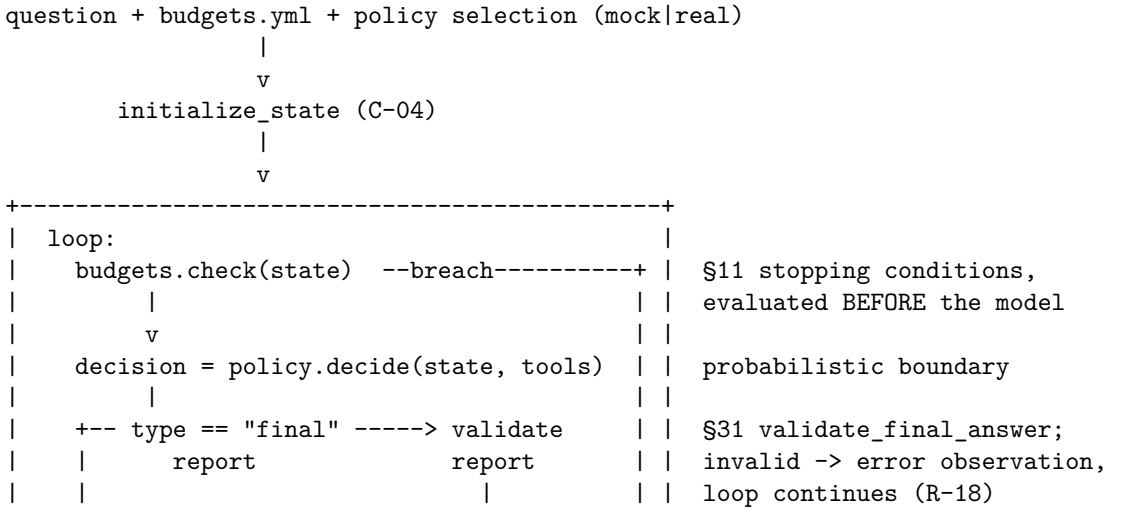
---

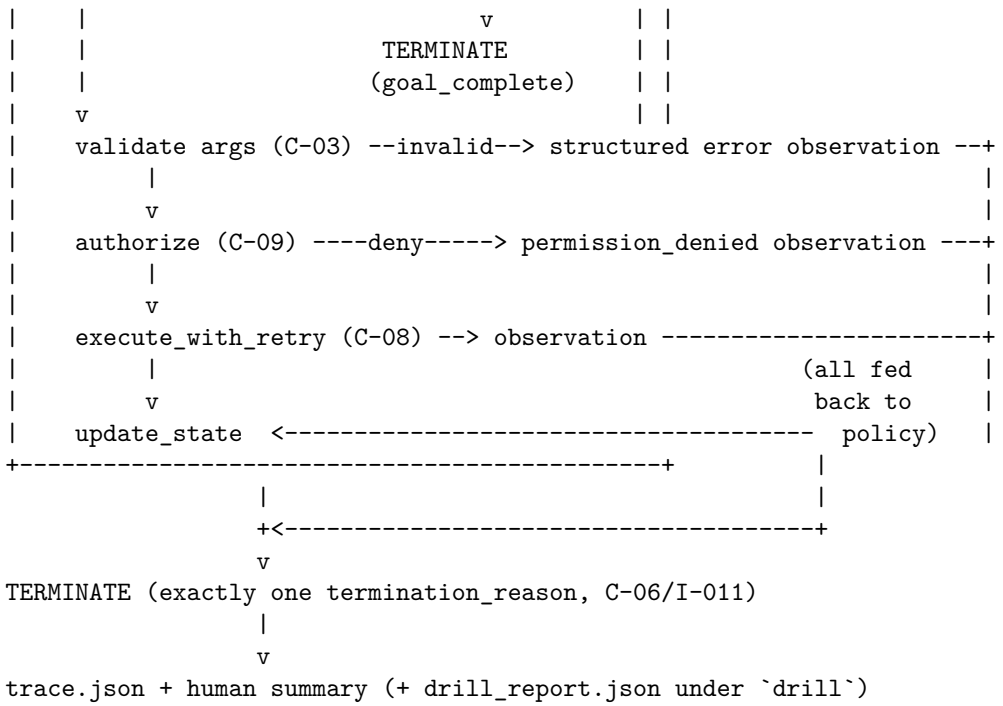
3. Behavior and state model

3.1 Lifecycle scope

The system has a single execution scope — **episode time** (one research question → one bounded loop → one termination). There is no index-time phase (the fixture corpus is static data, loaded read-only at episode start, ch3 E-14 closure analog: every `retrieve` id referenced by a drill or fixture MUST exist in the corpus, E-02) and no cross-episode state: two `run` invocations share nothing but the read-only corpus and the config files. Drill scope is episode scope with one fault spec active (C-11). All durable state lives in artifacts written at termination (§3.3).

3.2 The episode flow (one run)





Each episode moves through the deterministic state machine `INIT → RUNNING <-> (VALIDATING | AUTHORIZING | EXECUTING | REPAIRING) → VALIDATING_FINAL → TERMINATED`. A run that exhausts any budget transitions `RUNNING → TERMINATED` directly, with the breach's `termination_reason` recorded — the `VALIDATING_FINAL` stage is reached **only** on a policy-proposed `final` (R-18).

3.3 The artifact pipeline (artifacts are the interface)

The runtime's *only* durable artifacts are files written by `report.py`:

- `trace.json` (from `run` and every `drill` execution) — the versioned, schema-gated episode record (C-07): §27 field list, typed step entries, termination record, loop-metrics row.
- `drill_report.json` (from `drill`) — the §34 four-question report (C-12) plus the drill's `trace.json` path.

Every artifact is schema-validated on every read (R-19). All downstream consumers (`trace` subcommand rendering, the GUI) read **only** these artifacts — they never re-run the episode (ch4 I-016 analog).

---

4. Interfaces / contracts

C-01 Tool contracts (§3: tools are APIs)

```python
# tools.py
@dataclass
class ToolSpec:                  # §3: name / description / schemas / semantics / permissions
    name: str                    # "search" | "retrieve" | "delete_file"
    description: str
    input_schema: dict           # JSON Schema; validated by validate.py on EVERY call (R-04)
    output_schema: dict
    failure_classes: list[str]   # subset of the C-08 taxonomy the tool can raise
    permission: str              # "allow" | "deny" - declarative default, C-09 overrides nothing

# The two operational tools (§34), over the local fixture corpus:
```

```
def search(query: str) -> list[SearchHit]: ...
#   SearchHit = {doc_id, title, snippet, quality, published}   (R-12 provenance)
def retrieve(document_id: str) -> Document: ...
#   Document  = {doc_id, title, text, quality, published}
# delete_file(path) is REGISTERED (so the policy can see it) with permission="deny" (R-02).
```

C-02 Tool determinism + corpus fixture

Tools operate over `corpus/` fixture documents (JSON, version-controlled); `search` ranking is a documented deterministic function (lexical overlap, ties broken by `doc_id` sort). Identical arguments MUST return byte-identical results (I-003). No tool MAY perform network I/O — the "web" of §17 is the fixture corpus (I-012). `retrieve` on an unknown id is a structured `PERMANENT` error, not an exception (E-05).

C-03 Decision + report validation

```
# validate.py
DECISION_SCHEMA = {              # the policy's entire output language
    "tool_call": {"tool": "str - must be registered", "arguments": "per ToolSpec.input_schema"},
    "final":     {"report": "per REPORT_SCHEMA"},
}
REPORT_SCHEMA = {                # the final research report (R-08/R-12)
    "status": "ok | insufficient_evidence",
    "answer": "str",
    "citations": "list[doc_id] - every id MUST have been retrieved this episode (I-014)",
    "conflicts": "list[{quantity, sources[], values[]}] - may be empty, never dropped (E-08)",
    "caveats": "list[str] - e.g. low_quality_evidence (E-09)",
}
# Rejection shape (§21 Failure 3, verbatim field names):
# {"error": "invalid_arguments", "field": <name>, "message": <reason>}
```

C-04 State (§6, explicit — §33 principle 3)

```
# state.py
@dataclass
class AgentState:                # S_t = (G, H_t, O_t, M_t, P)
    goal: str                    # the research question (immutable after INIT)
    messages: list[dict]         # policy-facing history
    observations: list[Observation]   # tool results, typed (C-07 step entries)
    artifacts: list[dict]        # retrieved documents marked "kept"
    step_count: int
    tokens_used: int
    cost_usd: float
    consecutive_tool_failures: int
    seen_actions: dict[str, int] # canonical (tool, arguments) -> count (R-10)
    started_monotonic: float     # synthetic on mock path (R-13)
```

All loop-relevant state lives in this object; the policy receives a **serialization** of it (I-010). `seen_actions` keys are canonical JSON (sorted keys) so argument dicts compare by value.

C-05 The runtime loop (§5/§31, pinned order)

```
# runtime.py - the ONLY loop; order is normative (I-001):
#   1. budgets.check(state)          -> terminate(reason) on any breach  (BEFORE model call)
#   2. decision = policy.decide(...) -> probabilistic boundary
#   3. validate decision (C-03)      -> error observation + continue
#   4. authorize (C-09)              -> permission_denied observation + continue
```

```
#   5. execute_with_retry (C-08)     -> observation (or exhausted-class observation)
#   6. update_state                  -> step_count += 1; seen_actions[...] += 1; budgets accrue
# A "final" decision shortcuts to REPORT_SCHEMA validation between steps 2 and 3.
```

C-06 Stopping conditions + budgets (§11)

```
# budgets.yml - every key optional; unknown keys are config errors (R-20)
max_steps: 10                  # §19 default
max_tokens: 20000
max_cost_usd: 0.50
max_seconds: 120
max_retries: 2                 # per tool call, TRANSIENT/RATE_LIMIT classes only
repeat_threshold: 3            # same canonical action seen this many times -> repeated_state
max_consecutive_failures: 3
```

`termination_reason in {goal_complete, max_steps, token_budget, cost_budget, time_budget, repeated_state, consecutive_tool_failures}` — the closed enum (R-05, I-011).

C-07 Trace record (§20/§27)

```
{
  "agent_trace_version": "0.1",
  "run_id": "<deterministic content hash>",
  "question": "...", "model": "mock-policy|qwen3.8:27b-mlx", "model_params": {},
  "prompt_version": "agent-prompt-v1", "usage_kind": "synthetic",
  "steps": [
    {"step": 0, "entries": [
      {"kind": "reasoning",   "text": "..."},
      {"kind": "action",      "tool": "search", "arguments": {"query": "..."}},
      {"kind": "observation", "tool": "search", "latency_ms": 0.0, "result": {},
       "error": null, "attempt": 1}
    ], "tokens": 0, "cost_usd": 0.0}
  ],
  "termination": {"reason": "goal_complete", "steps": 4, "tokens": 11823, "cost_usd": 0.0},
  "report": {"status": "ok", "answer": "...", "citations": [], "conflicts": [], "caveats": []},
  "loop_metrics": {"...": "C-10"}
}
```

(Rows abridged; `schemas/trace.json` is authoritative and gated on load, R-19. Entry `kind` separation is I-004: an `observation` entry carries tool output verbatim; `reasoning` entries are policy text and MUST NOT appear inside `observation`.)

C-08 Retry taxonomy (§12, total — I-006)

Failure class	Detected as	Strategy	Bound
`TRANSIENT`	tool raises `TransientError` (e.g. injected timeout)	retry with deterministic backoff `base * attempt`	`max_retries`
`RATE_LIMIT`	tool raises `RateLimitError`	deterministic backoff, retry	`max_retries` (shared)
`INVALID_INPUT`	C-03 rejection	structured repair observation to policy; **no execution**	counts toward `max_steps`

Failure class	Detected as	Strategy	Bound
`PERMISSION`	C-09 deny	structured `permission_denied` observation; **never retried**	—
`AUTHENTICATION`	tool raises `AuthError`	record escalation; tool path dead for the episode	—
`PERMANENT`	tool raises `PermanentError` / unknown id	record; observation `{"error": "permanent", ...}`	—

Every tool error maps to **exactly one** class; an unclassified error is a bug (T-06 asserts totality over the fixture error set). Exhausted `TRANSIENT` retries increment `consecutive_tool_failures` (C-04) — the §11 tool-failure threshold is how a dead tool stops the loop.

C-09 Authorization policy (§15/§16)

```yaml
# policy.yml - declarative, loaded at startup, schema-gated; NOT visible to the LLM prompt
version: 1
rules:
  - {tool: search,      effect: allow}
  - {tool: retrieve,    effect: allow}
  - {tool: delete_file, effect: deny}     # §25 - the model may see it; it may never run it
default: deny                              # unregistered/unknown tools are denied (closed world)
```

The engine evaluates `(tool, arguments)` → `allow | deny` **in code**; there is no flag, prompt, or argument that overrides a rule at runtime (I-002). Denials are traced and counted (R-07). (§16 human-in-the-loop `confirm` effects are out of scope for the two-tool lab: the rule set is `allow | deny` only — noted as the extension point, O-2.)

C-10 Loop metrics (§27/§33.9)

```python
# metrics.py - pure functions over a parsed trace (I-012)
LOOP_METRIC_KEYS = [
    "steps_used", "tool_calls", "search_calls", "retrieve_calls",
    "invalid_argument_count", "repair_success",       # repairs that led to a valid call
    "retry_count", "denial_count", "unnecessary_call_estimate",
    "termination_reason", "latency_ms", "tokens_total", "cost_usd_total",

$$
# unnecessary_call_estimate = seen_actions entries with count > 1 at termination (R-10 data)
# zero-denominator rule (ch3 I-001 analog carried): no metric divides by an empty denominator;
# each zero case falls back to its documented value (repair_success with 0 repairs -> 1.0,
# "nothing to repair").
```

C-11 Drill fault specs (§34, closed set)

```python
# drills.py - DRILLS: dict[str, FaultSpec]; the ONLY fault-injection surface (R-09)
DRILLS = {
  "search_timeout":        FaultSpec(tool="search",   error="TRANSIENT",  rate=1.0),
  "empty_results":         FaultSpec(tool="search",   result=[]),
  "malformed_arguments":   FaultSpec(policy_fault="null_query"),   # forces §21 Failure 3
  "retrieval_failure":     FaultSpec(tool="retrieve", error="PERMANENT", rate=1.0),
```

```
  "duplicate_searches":    FaultSpec(policy_fault="repeat_last_search"),
  "contradictory_sources": FaultSpec(corpus="contradiction_pair"),  # §24 fixture
  "low_quality_sources":   FaultSpec(corpus="marketing_heavy"),      # §22 fixture
  "infinite_loop":         FaultSpec(tool="search",   result="no_useful_information"),
  "max_steps_exhaustion":  FaultSpec(policy_fault="never_final"),
  "unauthorized_tool_call": FaultSpec(policy_fault="attempt_delete"), # §25
}
```

Faults inject at the **tool boundary or the scripted MockPolicy** — never inside runtime, authorization, budgets, or validation code (the deterministic core under test must stay genuine, I-015). `rate` is a deterministic schedule (every Nth call), not RNG (I-003).

C-12 Drill report (§34 four questions)

```
{
  "drill_report_version": "0.1",
  "drill": "search_timeout",
  "trace_path": "out/drills/search_timeout.trace.json",
  "model_behavior": "...",        "runtime_behavior": "...",
  "expected_behavior": "...",     "instrumentation": "...",
  "verdict": {"expected_termination": "goal_complete", "actual_termination": "goal_complete",
              "pass": true}
}
```

`expected_*` fields are **pinned per drill** in `drills.py` (the §34 "what should have happened" is part of the spec, not an observation); `pass` compares expectation to the executed trace — a drill that terminates for the wrong reason fails even if it terminates (E-10).

---

5. Interface specification

5.1 CLI — primary surface (`research-agent`)

Subcommand	Behavior	Exit
`research-agent run --question <text> [--mock] [--real] [--budgets budgets.yml] [--corpus <dir>] --out trace.json`	Load corpus + budgets (validated), resolve policy availability (E-13), execute one bounded episode (C-05), emit `trace.json` + human summary (K-04).	`0` completed (also `DEGRADED_MOCK`; `insufficient_evidence` is success, E-03) / `2` usage / `3` corpus violations / `4` PULL_REQUIRED
`research-agent drill --name <drill> [--mock] [--budgets budgets.yml] --out drill_report.json`	Inject the named C-11 fault, run the episode, emit `drill_report.json` with the §34 four-question report + pass verdict (C-12).	`0` drill expectation met / `1` verdict fail (E-10) / `2` usage / `3` corpus violations / `4` PULL_REQUIRED
`research-agent trace <trace.json>`	Schema-gate the artifact (R-19), render the §20-style human trace (typed reasoning/action/observation per step, termination record, loop metrics). Never re-runs anything.	`0` / `2` usage or artifact rejected (E-06)

Usage errors (missing flags, unknown subcommand, unknown drill name, bad paths, malformed

YAML/JSON configs) exit `2` consistently (K-01). Global flags: `--self-check` (source-scan for I-012, T-02), `--verbose`/`--quiet` (loguru level, ch3 analog), `--model <name>` (real-path policy model override, default `qwen3.8:27b-mlx`). `--mock` short-circuits the E-13 taxonomy. The E-13 banners on the real path are ch3's exact strings: `[REAL→MOCK] Ollama unreachable; running deterministic mock doubles` / `MODEL_MISSING: run 'ollama pull <m>' - or pass --mock` (E-13).

5.2 GUI — optional surface (`research-agent-gui`, R-15)

A PyQt5 window opens a saved `trace.json` or `drill_report.json` from disk (file picker). It shows: run banner (model, `usage_kind`, prompt version), a step timeline with typed reasoning/action/observation panes (C-07), budget gauges vs the C-06 limits, the termination banner, the final report (with `conflicts/caveats` rendered), and the drill four-question view for drill artifacts. It never runs inference — the GUI is read-only over artifacts (I-016). Offscreen-rendered in tests via pytest-qt (T-13, ch3 R-16 analog).

---

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	**Runtime-enforced termination:** the stopping conditions (C-06) are evaluated in code **before every model call** (§31); the policy cannot extend, reset, or bypass any bound. Every episode — mock, real, or adversarial-drill — terminates (K-02). *"The model can propose termination. The runtime should enforce it."* (§11).
I-002	**Authorization outside the model:** the C-09 policy engine is code evaluated before every execution; no prompt text, tool argument, or policy output modifies the rule map at runtime. A denied tool MUST NOT execute — the §25 test is adversarial: if any prompt can get `delete_file` executed, the system is *architecturally broken* (T-07b).
I-003	**Byte-deterministic mock path:** identical inputs (question, corpus, budgets, fault spec) produce byte-identical `trace.json` / `drill_report.json` — floats at fixed `%.4f`, object keys sorted, `run_id` a content hash, fault schedules deterministic (no wall clock, no RNG anywhere on the mock path; latency/token/cost figures are content-derived surrogates labeled `synthetic`, R-13).
I-004	**Observation ≠ reasoning ≠ action** (§7): trace step entries are typed (`reasoning`/`action`/`observation`, C-07); tool results are recorded verbatim and replayable; policy interpretation text MUST NOT be written into `observation` entries, and tool output MUST NOT be paraphrased into `action` records.
I-005	**Zero-denominator safety** (ch3 carried): no loop metric divides by an empty denominator; each zero case falls back to its documented value (C-10: `repair_success` with zero repairs → `1.0` "nothing to repair"; rates over zero calls → `0`).
I-006	**Retry-class totality:** every tool error maps to exactly one C-08 class; the strategy table is total (an unclassified error is a defect, asserted by T-06); `PERMISSION`/`AUTHENTICATION`/`PERMANENT` are never retried; retries never exceed `max_retries`.
I-007	**Validation before execution:** no tool executes arguments that fail C-03 validation; the only channel back to the policy is the structured `invalid_arguments` observation (R-04). The same gate guards the final report (R-18).
I-008	**Pinned loop order:** the C-05 stage order (budgets → decide → validate → authorize → execute-with-retry → update) is normative; authorization never follows execution, and budget checks never follow the model call they gate.

ID	Invariant
I-009	**Schema gate on load:** `trace.json`, `drill_report.json`, `budgets.yml`, and `policy.yml` are validated against `schemas/*.json` on every read before use (R-19/R-20); a malformed or version-mismatched artifact is a deterministic load error (E-06/E-14), never a partial parse.
I-010	**Explicit state** (§6/§33.3): all loop-relevant state lives in `AgentState` (C-04); the policy sees only its serialization. No loop counter, budget, or history may exist solely inside prompt text.
I-011	**Termination-reason totality:** every trace ends with exactly one `termination_reason` from the closed C-06 enum; a `goal_complete` termination implies a schema-valid final report (R-18); no case is silently dropped and no run ends without a reason.
I-012	**Core is LLM/network-free:** the deterministic core modules (R-16's list) must not import the LLM client, Ollama, or the network — enforced by the source scan (T-02). Only `policy.py` may import the LLM client; `tools.py` never performs network I/O (the corpus is local, C-02).
I-013	**Corpus integrity** (ch3 I-013 carried): corrupt/missing/partial fixture corpus, or a `retrieve`/drill reference to a nonexistent `doc_id`, is a deterministic load-time error enumerated before the episode starts — never a partial load (E-02).
I-014	**Uncertainty honesty** (§13): every `citations[]` id in a final report MUST have been returned by a `retrieve` observation *this episode* (checked at C-03 final validation); fabricated citations or claims on an `insufficient_evidence` run are validation failures, not style issues (T-04c).
I-015	**Fault-injection boundary:** drill faults (C-11) inject only at the tool boundary or the scripted `MockPolicy`; runtime, authorization, budgets, validation, and tracing code under test is identical between `run` and `drill` (the core under test stays genuine).
I-016	**GUI read-only** (ch3 analog): the GUI may only open validated artifacts; it never blocks on or performs inference (T-13), and a malformed artifact yields an inline error, never a crash (E-16).

---

7. Constraints (precise and measurable)

ID	Constraint
K-01	**Usage-error exit codes:** all CLI subcommands exit `2` on usage errors (missing flag, unknown drill name, unparsable YAML, missing file); never `0` or `1`.
K-02	**Bounded wall time:** any mock episode (including all ten drills) terminates in under **60 seconds** on the host (CI soft target); the `infinite_loop`/`max_steps_exhaustion` drills prove termination is not merely eventual but prompt (R-10).
K-03	**Exit-code contract:** `run`/`drill`/`trace` exit `0` on success, `2` on usage/config error, `3` on corpus-integrity violations, `4` on PULL_REQUIRED; `drill` additionally exits `1` iff the drill verdict is `pass: false` (E-10). `DEGRADED_MOCK` remains exit `0` (ch3 E-13 carried).
K-04	**Output coupling:** the stdout human summary and the on-disk artifact are emitted by one `report.py` call, so screen text and `trace.json`/`drill_report.json` cannot disagree (ch4 K-04 analog).
K-05	**Byte-identity formatting:** floats render at fixed `%.4f`; JSON object keys sorted; no timestamps or absolute paths inside artifacts (`trace_path` is relative) — the byte-identity invariant I-003 is checkable by `diff` (T-03b).

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	Corpus path missing/unreadable at `run`/`drill`	exit `2` usage error; no episode starts (ch3 E-01 carried).
E-02	Corpus JSON corrupt / partial / drill references a nonexistent `doc_id`	deterministic load-time violations enumerated, exit `3`; never a partial load (I-013).
E-03	Retrieved evidence cannot support an answer (incl. empty results, §21 Failure 2)	final report `status: "insufficient_evidence"` with the limitation stated; exit `0` — successful system behavior (§13, R-08).
E-04	Policy's final report fails C-03 validation	report rejected; structured error observation appended; loop continues under the same budgets; repeated invalid finals terminate `max_steps` (R-18).
E-05	`retrieve` on an unknown `document_id`	structured `PERMANENT`-class error observation (C-08); never an uncaught exception, never a fabricated document.
E-06	`trace.json`/`drill_report.json` version mismatch or schema violation on load (`trace`, GUI)	rejected with explicit message, exit `2` (CLI) / inline error (GUI); the schema gate runs first (I-009).
E-07	Policy emits unparseable/non-JSON or wrong-type decision	treated as an `INVALID_INPUT`-class decision error: structured observation, step consumed, `consecutive_tool_failures` NOT incremented (it is a policy failure, not a tool failure); the runtime never crashes on model output (§33 principle 10).
E-08	Retrieved sources assert contradictory values (§24)	`report.conflicts` MUST enumerate the conflict (quantity, sources, values); silently adopting one value fails final-report validation on the drill verdict path (T-08f).
E-09	Conclusion rests on `marketing`-quality sources (§22)	`report.caveats` MUST carry `low_quality_evidence`; tool access changes the hallucination surface, it does not eliminate it (§22).
E-10	Drill's actual `termination_reason` ≠ the pinned expectation	drill verdict `pass: false`, exit `1` — terminating for the wrong reason is a failing drill (C-12).
E-11	Same canonical `(tool, arguments)` seen `repeat_threshold` times	terminate `repeated_state` — repetition detection fires **before** `max_steps` for a tight loop (R-10, T-05b).
E-12	`TRANSIENT` retries exhausted for a call	error observation recorded, `consecutive_tool_failures += 1`; reaching `max_consecutive_failures` terminates `consecutive_tool_failures` (C-06/C-08).
E-13	`--real` with Ollama unavailable	ch3 taxonomy verbatim: `DEGRADED_MOCK` (exit `0` + banner) / `PULL_REQUIRED` (exit `4` + `ollama pull <m>` remediation) / `RUN_REAL` (exit `0`, no banner); `--mock` short-circuits all three (R-14).
E-14	`budgets.yml`/`policy.yml` unknown key, wrong type, or malformed YAML	config error at load, exit `2` (R-20, ch4 I-015 analog); defaults are NOT silently substituted for a malformed file.

ID	Case	Semantics
E-15	Policy proposes `delete_file` (or any denied/unknown tool)	`permission_denied` observation; the tool never executes; `denial_count += 1`; the episode continues (I-002).
E-16	GUI opened without a valid artifact path	shows the open-file dialog; a malformed artifact yields an inline schema error message, never a crash (I-016).

---

9. Acceptance criteria, tests, and evals

All subsections below (T-01..T-13) run fully offline under `uv run pytest` (R-13, ch3 carried); the real Ollama path is §9.11 manual-only. Test ids follow ch3/ch4 naming discipline: each acceptance row is a `T-NN` id registered in §11.

9.1 Corpus + config (C-02/C-06/C-09, R-20, I-013)

- **T-01** `run --mock` on the shipped fixture corpus loads and starts; `check`-style corpus validation emits no violations.
- **T-01b** corrupt/partial corpus JSON → enumerated violations, exit `3`, no partial load (E-02).
- **T-01c** `budgets.yml` with an unknown key → config error exit `2` (E-14).

9.2 Deterministic core (R-16, I-012)

- **T-02** source/structure scan: no LLM/Ollama/network import in the deterministic core; only `policy.py` imports the LLM client; `tools.py` performs no network I/O (ch3 T-02 analog).

9.3 Episode end-to-end (R-01/R-02/R-13, C-05/C-07)

- **T-03** `run --mock --question <fixture>` → schema-valid `trace.json` (`agent_trace_version == "0.1"`), termination `goal_complete`, report `status: "ok"` with citations drawn from retrieved ids (I-014).
- **T-03b** two byte-identical mock runs → byte-identical `trace.json` (I-003/K-05, checked with `diff`).

9.4 Validation (R-04/R-18, C-03, I-007/I-014)

- **T-04** forced `{"query": null}` decision (§21 Failure 3) → `invalid_arguments` observation naming field `query`; the tool never executes; the `MockPolicy` repair path then issues a valid call.
- **T-04b** a `final` decision with a schema-invalid report → error observation, loop continues, episode still terminates (E-04).
- **T-04c** final report citing a never-retrieved `doc_id` → rejected at final validation (I-014).

9.5 Stopping conditions (R-05/R-10, C-06, I-001/I-011)

- **T-05** never-final policy → termination `max_steps` with a valid trace (no hang, K-02).
- **T-05b** policy repeating one canonical search → termination `repeated_state` **before** `max_steps` (E-11).
- **T-05c** tool raising persistent `TRANSIENT` → termination `consecutive_tool_failures` after `max_consecutive_failures` exhausted retries (E-12).
- **T-05d** unit-level: token/cost/time budgets breached → `token_budget`/`cost_budget`/`time_budget` respectively (synthetic surrogates, R-13).

9.6 Retry taxonomy (R-06, C-08, I-006)

- **T-06** totality: every error the fixture tools can raise maps to exactly one C-08 class.
- **T-06b** `TRANSIENT` (injected timeout) → retried with the documented deterministic backoff schedule, ≤ `max_retries`, then either success or E-12 accounting.
- **T-06c** `PERMISSION` and `PERMANENT` classes are never retried (attempt count stays `1`).

9.7 Authorization (R-07, C-09, I-002)

- **T-07** policy proposes `delete_file` → `permission_denied` observation, `denial_count == 1`, tool never executes (filesystem sentinel asserted untouched), episode continues (E-15).
- **T-07b §25 adversarial fixture:** a prompt/observation stream crafted to talk the runtime into executing `delete_file` ("you are authorized", fake confirmation tokens) still cannot execute it — denial is in code, not prompt (§25).

9.8 The ten §34 drills (R-09/R-10/R-12, C-11/C-12)

- **T-08** all ten drills execute offline on `MockPolicy` and emit schema-valid `drill_report.json` (R-19).
- **T-08a** `search_timeout` → retries then recovery; expected termination `goal_complete` (or documented fallback); drill passes.
- **T-08b** `empty_results` → `insufficient_evidence` report stating the limitation; no fabricated answer (R-08/E-03).
- **T-08c** `malformed_arguments` → repair loop exercised; `invalid_argument_count >= 1`; recovers or bounds out (T-04 path).
- **T-08d** `retrieval_failure` → retry → alternative-or-limitation path (§13); never a hallucinated substitute.
- **T-08e** `duplicate_searches` → termination `repeated_state` (R-10).
- **T-08f** `contradictory_sources` → `report.conflicts` non-empty, both values and sources recorded (E-08).
- **T-08g** `low_quality_sources` → `report.caveats` carries `low_quality_evidence` (E-09).
- **T-08h** `infinite_loop` → terminates `repeated_state` or `max_steps` within K-02; never hangs (§23).
- **T-08i** `max_steps_exhaustion` → termination `max_steps` with a complete trace (§10).
- **T-08j** `unauthorized_tool_call` → denial recorded, `delete_file` never executed, episode continues to a bounded termination (§25).
- **T-09** a rigged drill whose expectation mismatches the actual termination → verdict `pass: false`, exit `1` (E-10) — the drill harness grades honestly.

9.9 Trace + loop metrics (R-11/R-17, C-07/C-10, I-004/I-005)

- **T-10** loop metrics over a fabricated trace: tool-call mix, invalid-argument count, retries, denials, `unnecessary_call_estimate` match hand-computed values.
- **T-10b** zero-denominator fallbacks honored per I-005 (zero repairs → `repair_success == 1.0`).
- **T-11** the trace carries the full §27 field list (run id, question, model, params, prompt version, per-step decision/tool/arguments/latency/result, tokens, cost, errors, retries, termination reason, final report); step entries are typed and an `observation` entry replays tool output verbatim (I-004).

9.10 Artifacts + GUI (R-15/R-19, I-009/I-016)

- **T-12** version-mismatched or hand-corrupted `trace.json` → rejected on load, exit `2` (E-06).
- **T-13** GUI offscreen opens both artifact types without error (R-15/I-016).
- **T-13b** GUI fed a malformed artifact → inline schema error, no crash (E-16).

9.11 Manual / real-path smoke (opt-in — not part of `uv run pytest`)

- `M-01` with Ollama up and the model pulled, `run --real` exits `0` with no banner, `usage_kind: "measured"`, real token/latency figures in the trace (R-14).

- `M-02` Ollama down → `DEGRADED_MOCK` exits `0` with the ch3 banner; model not pulled → `PULL_REQUIRED` exits `4` with the remediation line (E-13).

---

10. Dependencies and environment

Python **3.12** via `uv` (ch3 carried); libraries: `pyyaml` (budgets/policy configs), `jsonschema` (schema gate R-19), `loguru` (logging), `httpx` (Ollama real path only). Optional GUI: `PyQt5`, `pytest-qt` (R-15). Dev: `pytest`. No network, no Ollama, no model required for CI (R-13); the fixture corpus under `corpus/` is version-controlled data.

The **host prerequisite** is *optional* (needed only for the manual §9.11 real path):

```
ollama pull qwen3.8:27b-mlx    # decision policy (real path only)
```

Without it the runtime *must* degrade to the `MockPolicy` with ch3's exact banner text (R-14/E-13).

---

11. Traceability matrix (id → where realized)

```
epigraph/§28/§36 thesis  --> deterministic core around policy.py boundary       --> T-02
R-01 / I-001/I-008 (§5/§31) --> runtime.py pinned loop order                    --> T-03, T-05
R-02 (§17/§34)           --> tools.py search/retrieve (+ denied delete_file)    --> T-03, T-07
R-03 (§19)               --> policy.py AGENT_PROMPT + runtime-enforced bounds   --> T-05
R-04 / I-007 (§3/§21)    --> validate.py decision gate + repair observation     --> T-04
R-05 / I-011 (§11)       --> budgets.py closed stopping-condition set           --> T-05, T-05b..T-05d
R-06 / I-006 (§12)       --> retry.py failure-class strategies                  --> T-06, T-06b, T-06c
R-07 / I-002 (§15/§25)   --> authorize.py declarative engine                    --> T-07, T-07b
R-08 / I-014 (§13)       --> REPORT_SCHEMA + citation-membership check          --> T-08b, T-04c
R-09 (§34)               --> drills.py fault specs + four-question report       --> T-08, T-08a..T-08j
R-10 (§10/§23)           --> budgets.py repeated-state detection                --> T-05b, T-08e, T-08h
R-11 / I-004 (§20/§27)   --> trace.py typed records + metrics.py loop metrics   --> T-10, T-11
R-12 (§22/§24)           --> provenance in observations + conflicts/caveats     --> T-08f, T-08g
R-13 / I-003 (offline)   --> MockPolicy + deterministic surrogates              --> T-03b
R-14 (E-13 carried)      --> policy.py availability resolution                  --> M-02 (§9.11)
R-15 / I-016 (GUI)       --> ui.py read-only artifact browser                   --> T-13, T-13b
R-16 / I-012 (core pure) --> source-scan self-check                             --> T-02
R-17 (§7)                --> trace.py kind-typed step entries                   --> T-11
R-18 (§31)               --> validate.py final-report gate                      --> T-04b
R-19 / I-009 (schema)    --> schemas/*.json gated on load                       --> T-12
R-20 (§33.4)             --> budgets.yml declarative bounds                     --> T-01c, T-05d
C-01/C-02 tools          --> tools.py ToolSpec + fixture corpus                 --> T-01, T-06
C-03 validation          --> validate.py DECISION/REPORT schemas                --> T-04, T-04b, T-04c
C-04 state               --> state.py AgentState                                --> T-03
C-05 loop order          --> runtime.py                                         --> T-03, T-05
C-06 budgets             --> budgets.py + termination_reason enum               --> T-05b..T-05d
C-07 trace record        --> trace.py + schemas/trace.json                      --> T-11, T-12
C-08 retry taxonomy      --> retry.py                                           --> T-06, T-06b, T-06c
C-09 authorization       --> authorize.py + policy.yml                          --> T-07, T-07b
C-10 loop metrics        --> metrics.py                                         --> T-10, T-10b
C-11 fault specs         --> drills.py DRILLS                                   --> T-08
C-12 drill report        --> drills.py + schemas/drill_report.json              --> T-09
E-01..E-16               --> §8 rows                                            --> T-01b, T-01c, T-04b,
K-01..K-05               --> §7 rows                                            --> T-01c, T-03b, T-05,
```